Reporte Técnico de Actividades Práctico-Experimentales Nro. 4

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	- Anthony Yaguana - Darwin Correa - Derick Vargas - Bryan Troya
Asignatura	Sistemas Digitales
Ciclo	5
Unidad	1 – Microcontroladores para sistemas embebidos
Resultado de aprendizaje de la unidad	R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	4
Título de la Práctica	Uso de interrupciones en un sistema de alerta doméstica
Nombre del Docente	Luis Darío Sinche C.
Fecha	6 de noviembre de 2026
Horario	Viernes 11h30 a 13h30
Lugar	Laboratorio de computación (simulación en Wokwi y prueba en la placa ESP32 del grupo)
Tiempo planificado en el Sílabo	3 horas (APE, semana 4): práctica presencial de menos de 1 h 30 min; el informe se termina fuera de clase
Placa	ESP32 DevKit V1 de 30 pines (módulo ESP32-WROOM-32)

2. Objetivo(s) de la Práctica

Objetivo. Construir un sensor de puerta con un pulsador y un LED que detecte la apertura mediante una interrupción externa del ESP32, y comprobar experimentalmente por qué la interrupción no pierde eventos que el sondeo sí pierde y por qué hace falta el antirrebote.

Al terminar, la pareja podrá:

- Configurar una interrupción externa con attachInterrupt() y el flanco correcto.
- Escribir una ISR correcta: corta, con IRAM_ATTR, variables volatile y patrón bandera.
- Explicar con sus propias mediciones la diferencia entre sondeo e interrupción y el efecto del rebote.

3. Materiales, Reactivos, Equipos y Herramientas

Software (cada estudiante):

- Wokwi (wokwi.com), en el navegador, sin instalar nada.
- Arduino IDE con el paquete esp32 by Espressif Systems versión 3.x, sin librerías externas, para la Parte C.
- Cuenta de GitHub y acceso al EVA, para la Parte D.

Hardware (por grupo, solo para la Parte C):

Cantidad	Componente
1	ESP32 DevKit V1 de 30 pines + cable USB de datos (no solo de carga)
1	Protoboard
1	Pulsador (botón) de 4 patas
1	LED rojo de 5 mm
1	Resistencia de 330 Ω
Varios	Cables jumper macho-macho

4. Procedimiento / Metodología Ejecutada

Primero se leyó y entendió el fundamento teórico, posteriormente se desarrolló la práctica en las siguientes partes.

PARTE A

- Se ingresó a la plataforma de Wokwi y se creó un proyecto nuevo seleccionando la placa ESP32 DevKit V1.
- Se armó el circuito en Wokwi conforme al Anexo A.
- Se inició la simulación y se presionó el pulsador 10 veces con toques cortos y rápidos (en un lapso de 10 segundos).
- Se observó el Monitor Serie y se comprobó que las aperturas reportadas fueron mucho menores a las 10 pulsaciones realizadas se registraron 1 y 2 aperturas en los dos intentos.
- Se detuvo la simulación, se reinició el experimento y se repitió la prueba una vez más para confirmar el resultado.
- Se tomaron capturas del Monitor Serie y se anotaron los datos en la Tabla A.

PARTE B

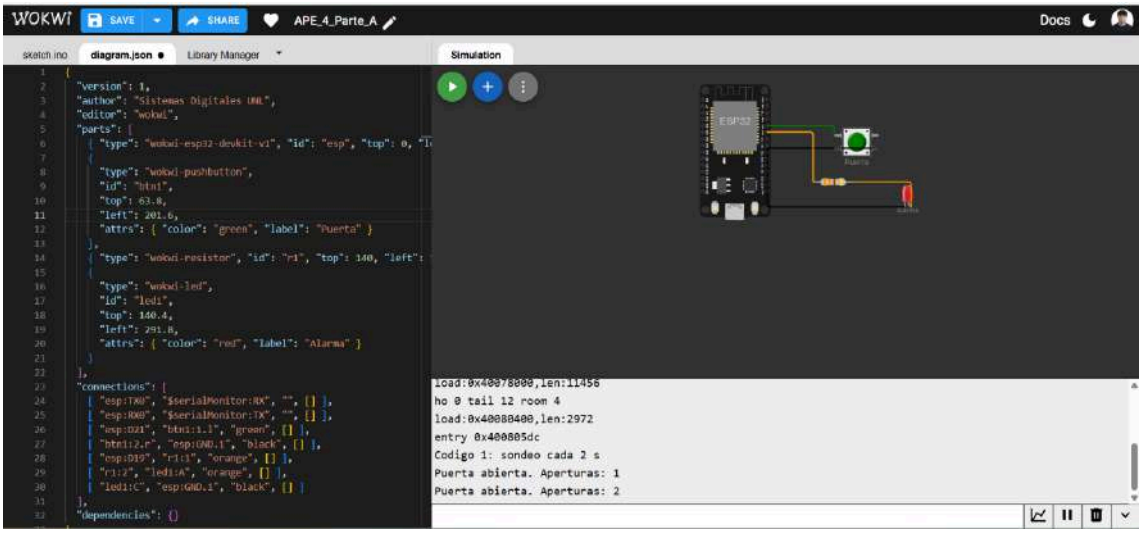
- En el mismo proyecto, se reemplazó el contenido de sketch.ino por el Código 2, con el mismo diagram.json.
- Se cambió la constante a T_REBOTE = 0 (antirrebote desactivado), se ejecutó la simulación y se presionó el pulsador 10 veces.
- Se anotó en la tabla B el último valor de eventos que muestra el monitor serie.
- Se volvió a T_REBOTE = 50 y se epitió los pasos B2 y B3.
- Se tomo una captura de pantalla para cada valor de T_REBOTE.

PARTE C

- Con el USB desconectado, se armó en la protoboard el pulsador entre D21 y GND (patas en diagonal) y el LED con su resistencia de 330 Ω entre D19 y GND.
- Se conectó la placa por USB. En Arduino IDE se instaló el core esp32 (v3.3.12) y se seleccionó ESP32 Dev Module en el puerto COM5.
- Se cargó el Código 2 con T_REBOTE = 0.
- Se abrió el Monitor Serie a 115200 baudios y se presionó EN para ver el mensaje inicial.
- Se presionó el pulsador 10 veces, con toques rápidos, y se anotó el último valor de «Eventos». Se presionó EN y se repitió.
- Se cambió a T_REBOTE = 50, se volvió a cargar el código y se repitieron los dos intentos.
- Se tomaron capturas del Monitor Serie de cada intento y una foto del montaje funcionando.

5. Resultados

Parte A:



Intento 1: 10 pulsaciones

```
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
Codigo 1: sondeo cada 2 s
Puerta abierta. Aperturas: 1
```

Intento 2: 10 pulsaciones

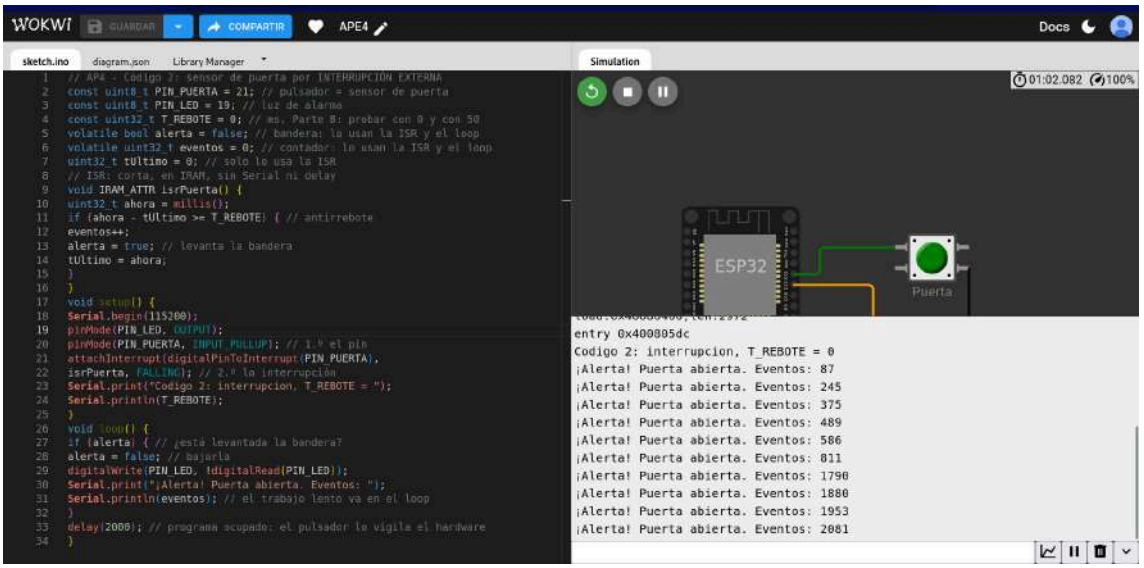
```
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
Codigo 1: sondeo cada 2 s
Puerta abierta. Aperturas: 1
Puerta abierta. Aperturas: 2
```

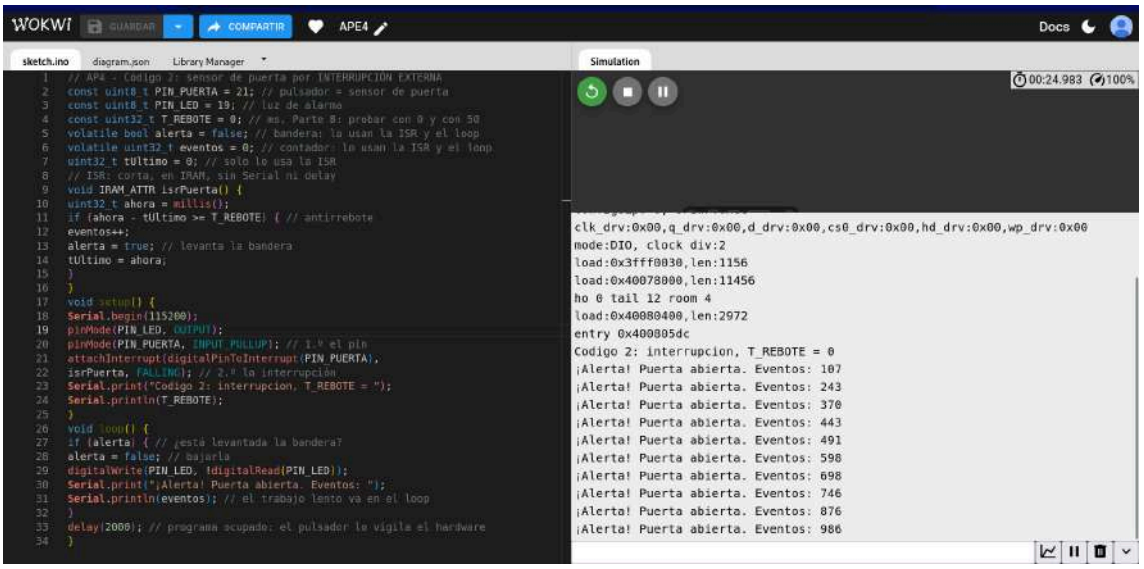
Tabla A. Sondeo: aperturas reportadas con 10 pulsaciones

Intento	Pulsaciones realizadas	Aperturas reportadas
1	10	1
2	10	2

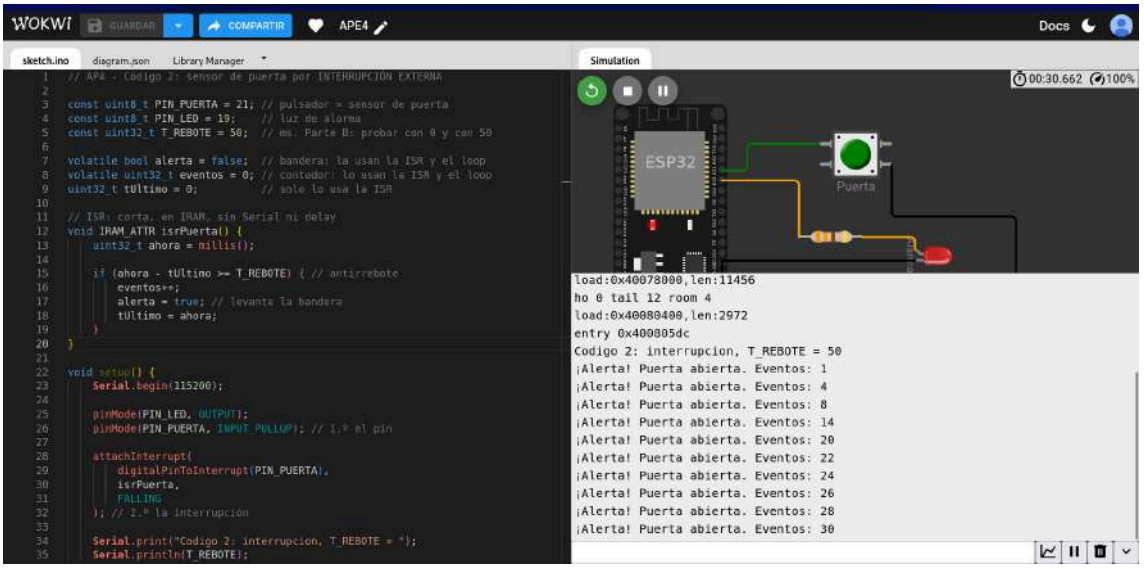
Parte B:

T_REBOTE = 0





T_REBOTE = 50



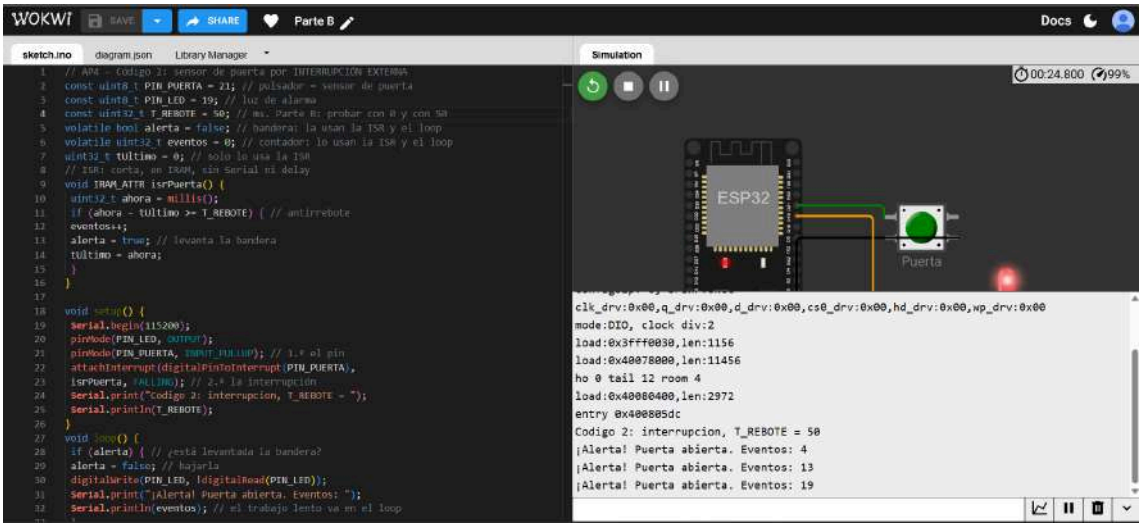


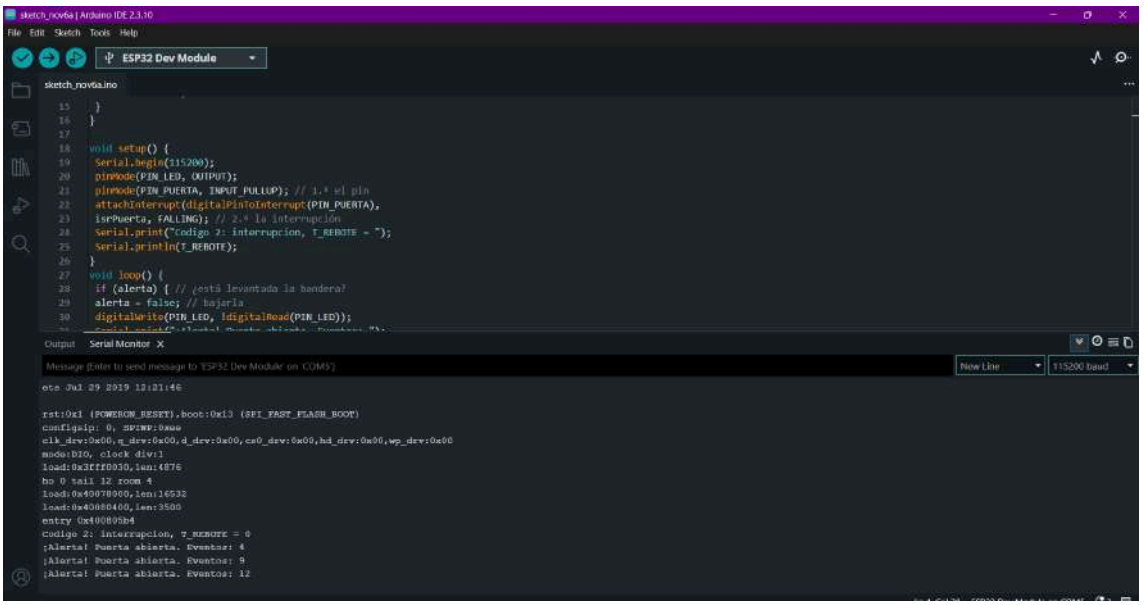
Tabla B

T_REBOTE (ms)	Eventos, intento 1	Eventos, intento 2	¿Más, menos o igual a 10?
0	2081	986	Más que 10
50	30	19	Más que 10

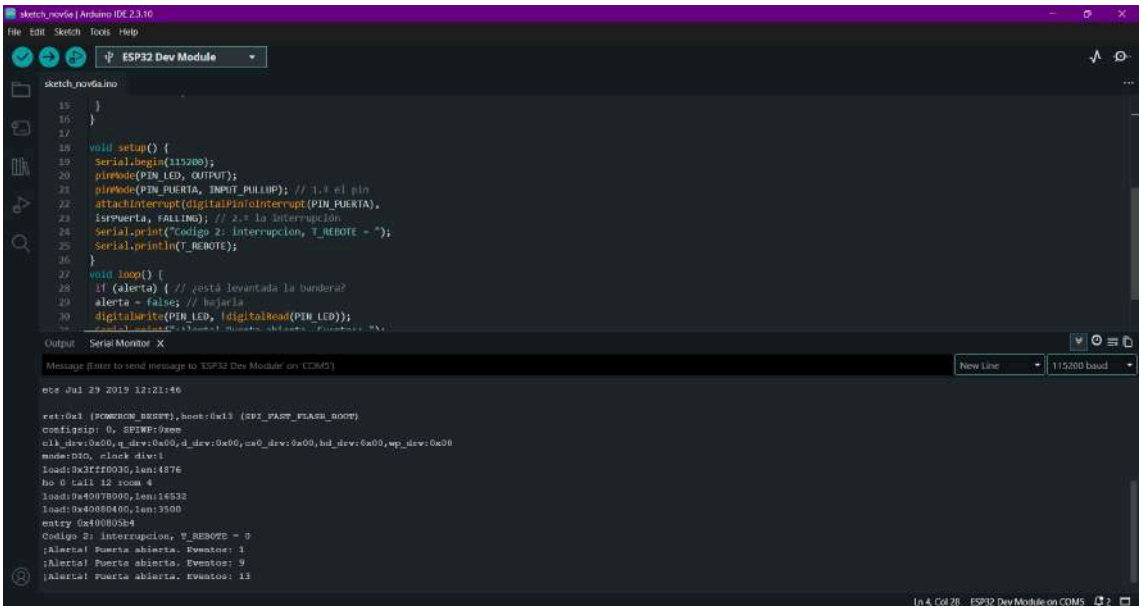
Parte C:

T_REBOTE = 0

Intento 1:

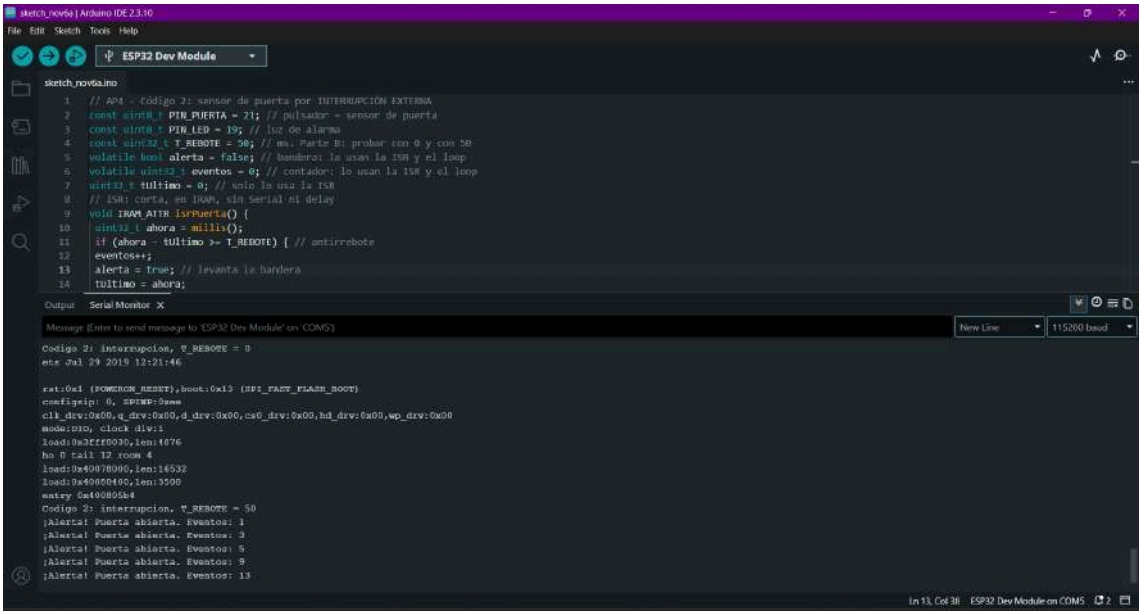


Intento 2:

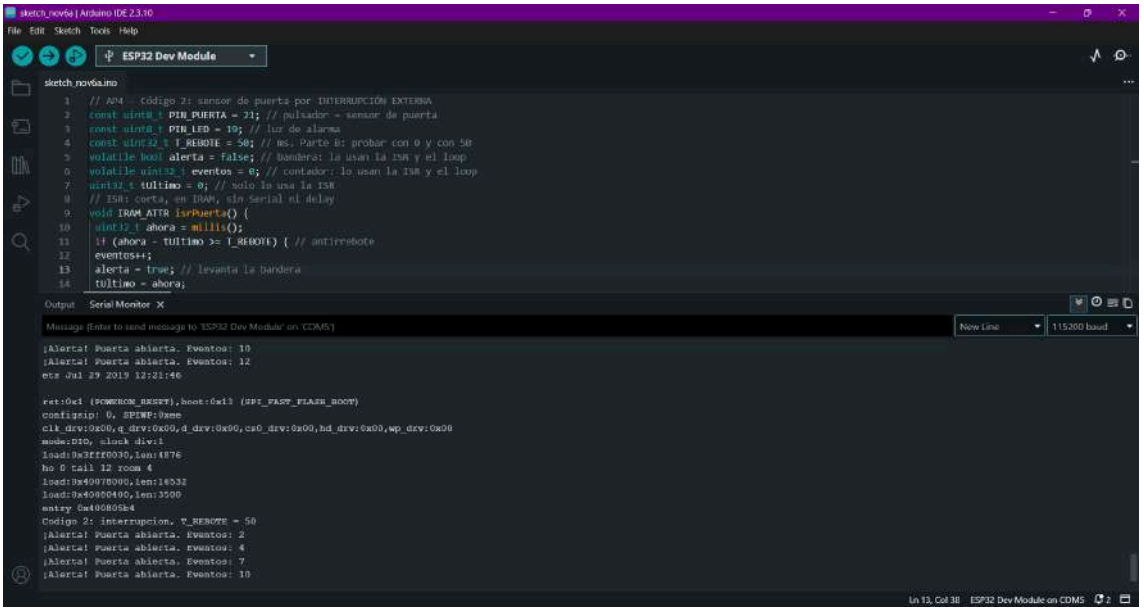


T_REBOTE = 50

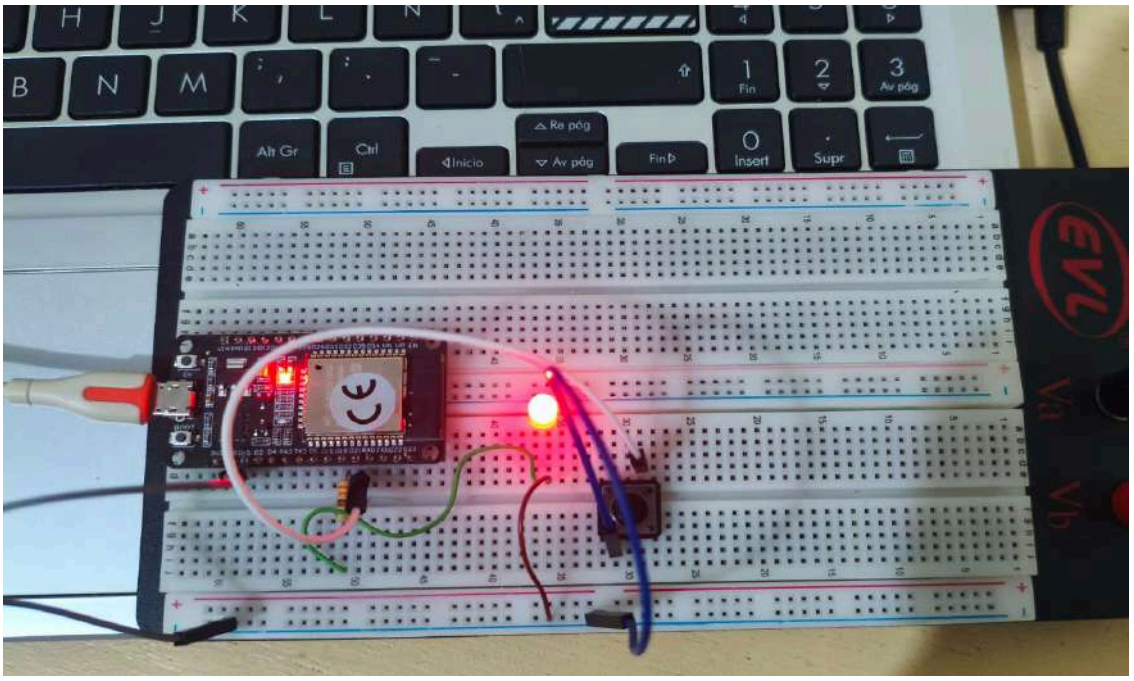
Intento 1:



Intento 2:



Placa funcionando:



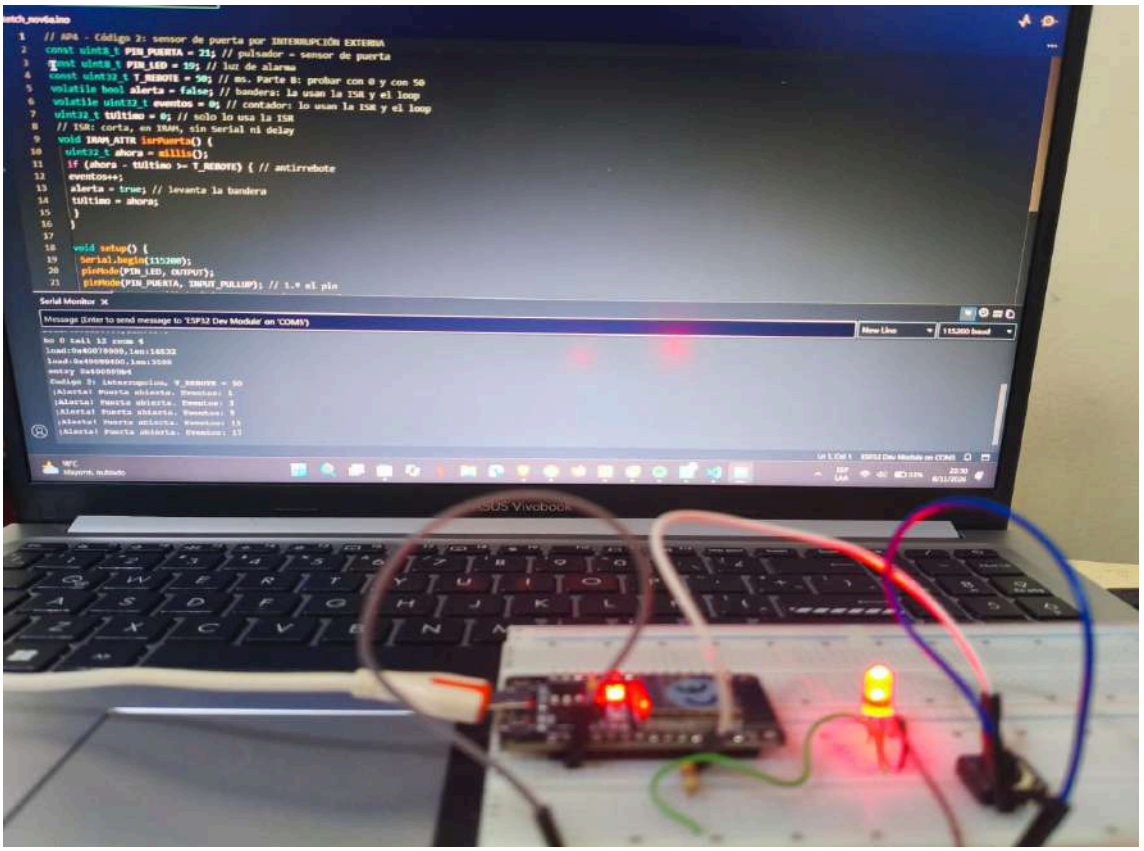

// RP4 - Código 2: sensor de puerta por INTERRUPCIÓN EXTERNA
const uint8_t PIN_PUERTA = 21; // pulsador = sensor de puerta
const uint8_t PIN_LED = 19; // luz de alarma
const uint32_t T_REBOTE = 50; // ms. Parte B: probar con 0 y con 50
volatile bool alerta = false; // bandera: la usan la ISR y el loop
volatile uint32_t eventos = 0; // contador: lo usan la ISR y el loop
uint32_t tUltimo = 0; // solo lo usa la ISR
// ISR: corta, en IRAM, sin Serial ni delay
void IRAM_ATTR isrPuerta() {
uint32_t ahora = millis();
if (ahora - tUltimo >= T_REBOTE) { // antirrebote
eventos++;
alerta = true; // levanta la bandera
tUltimo = ahora;
}
}
void setup() {
Serial.begin(115200);
pinMode(PIN_LED, OUTPUT);
pinMode(PIN_PUERTA, INPUT_PULLUP); // 1.º el pin
Serial Monitor
Message (Enter to send message to 'ESP32 Dev Module' on 'COM5')
New Line
115200 baud
ASUS Vivobook

Tabla C

T_REBOTE (ms)	Eventos, intento 1	Eventos, intento 2	Observaciones
0	12	13	Más eventos que pulsaciones en ambos intentos: los rebotes mecánicos disparan la ISR. En una prueba adicional se llegó a 21 eventos con 10 pulsaciones. El contador sigue subiendo durante el delay(2000) (saltos 1 → 9 → 13).
50	13	10	En el intento 2 el conteo coincidió con las 10 pulsaciones. En el intento 1 hubo 3 eventos extra, atribuibles al rebote al soltar el botón, que llega más de 50 ms después del flanco aceptado.

6. Preguntas de Control

- **Con el Código 1, ¿por qué se perdieron aperturas? Explíquenlo con lo que hace el loop() durante el delay(2000).**

  Se perdieron aperturas porque el Código 1 utiliza sondeo (digitalRead) y tiene un delay(2000). Cuando presioné el botón varias veces rápido, el programa estaba "congelado" esperando esos 2 segundos. Como el ESP32 no revisa el pin durante ese tiempo, los toques rápidos ocurrieron sin que el microcontrolador se diera cuenta, por lo que no los sumó al contador.

- **El Código 2 también tiene un delay(2000). ¿Por qué esta vez no se pierden aperturas? ¿Quién vigila el pin mientras el loop() espera?**

  Porque el Código 2 utiliza una interrupción externa que permite detectar las pulsaciones aunque el programa esté ejecutando el delay(2000). El hardware del ESP32 vigila el pin GPIO21 y ejecuta la función de interrupción (ISR) cuando detecta el flanco de bajada. De esta manera, los eventos pueden registrarse sin esperar a que termine el delay().

- **¿Por qué el contador marcó más de 10 eventos con T_REBOTE = 0? ¿Qué cambió con 50 ms?**

  Con T_REBOTE = 0, el contador registró más eventos porque el pulsador produce rebotes eléctricos que generan varias interrupciones durante una sola pulsación. Al utilizar 50 ms, se aplica un filtro que ignora los cambios producidos dentro de ese intervalo. En nuestras pruebas, el contador se redujo, aunque todavía registró 20 eventos con 10 pulsaciones.

- **En el Código 2, ¿por qué alerta y eventos son volatile y tUltimo no?**

Las variables alerta y eventos se declaran como volatile porque son utilizadas tanto por la función de interrupción (ISR) como por el loop(). Esto permite que el programa consulte sus valores actualizados. En cambio, tUltimo no necesita volatile porque solamente se utiliza dentro de la ISR para controlar el tiempo de antirrebote.

- **¿Por qué el Serial.print está en el loop() y no en la ISR? ¿Qué podría pasar si se mueve a la ISR?**

  Porque la función de interrupción debe ejecutarse rápidamente y realizar únicamente operaciones sencillas. Serial.print puede tardar demasiado en ejecutarse. Si se coloca dentro de la ISR, podría causar bloqueos, errores o reinicios del ESP32. Por eso, la ISR activa una bandera y el mensaje se imprime posteriormente desde el loop()

- **Si se usara RISING en lugar de FALLING, ¿en qué momento se encendería la alarma? ¿Por qué?**

  La alarma se activaría al soltar el pulsador en lugar de hacerlo al presionarlo. Esto sucede porque se utiliza INPUT_PULLUP, donde el pin tiene un valor HIGH cuando el pulsador está suelto y LOW cuando está presionado. RISING detecta el cambio de LOW a HIGH, mientras que FALLING detecta el cambio de HIGH a LOW.

7. Conclusiones

- Se comprobó experimentalmente que al usar el Código 1 (sondeo con un delay(2000)), de 10 pulsaciones rápidas realizadas, el sistema solo detectó 1 y 2 aperturas respectivamente. Esto demuestra que si el microcontrolador está ocupado ejecutando otras tareas, los cambios de estado en los pines se pierden por completo.

- Se logró configurar una interrupción externa en el ESP32 mediante la función attachInterrupt() y el flanco FALLING, permitiendo detectar las pulsaciones del botón sin depender de la ejecución continua del programa principal.

- Se comprendió la importancia de utilizar una ISR corta con IRAM_ATTR, variables volatile y una bandera para registrar los eventos. Esto permitió separar la detección de las pulsaciones de las tareas de encender el LED y mostrar los mensajes en el Monitor Serie.

- Se observó que el método de sondeo puede perder pulsaciones rápidas debido al delay(2000), mientras que las interrupciones permiten registrarlas aunque el

programa esté ocupado. Además, se comprobó la importancia del antirrebote, ya que con T_REBOTE = 50 se registraron 20 eventos para 10 pulsaciones, evidenciando que todavía existieron detecciones adicionales.

- En la placa real, sin antirrebote el contador superó siempre las 10 pulsaciones (entre 12 y 21 eventos), mientras que con T_REBOTE = 50 ms se obtuvo 10 y 13; el antirrebote reduce los conteos falsos, pero el rebote de un pulsador físico es variable y parte de él (al soltar) puede superar la ventana de 50 ms.

8. Bibliografía

- Diapositivas de la semana 4

9. Anexos

- Repositorio: https://github.com/Anthony-YP/Sistemas_Digitales.git
- Woki:
  - Parte A: https://wokwi.com/projects/479986770085532673
  - Parte B: https://wokwi.com/projects/480002507880152065